

Single cell Assay for Transposase-Accessible Chromatin sequencing (scATAC-seq)

For this work, we will use data from the article by Kumegawa et al. (2022) entitled: "[GRHL2 motif is associated with intratumor heterogeneity of cis-regulatory elements in luminal breast cancer](#)".

In this paper, Kumegawa et al. analyze chromatin accessibility profiles of more than 10.000 cells from 16 breast cancer patients including luminal, luminal-HER2, HER2+ and 3 triple-negative subtypes. Using this profiling process, they classified cells into cancer cells and tumor microenvironment, allowing to highlight the heterogeneity of disease-related pathways. Moreover, they identified the GRHL2 transcription factor which cooperated with FOXA1 to initiate endocrine resistance and that GRHL2 binding elements potentially regulate genes associated with endocrine resistance, metastasis, and poor prognosis in patients that received hormonal therapy.

The scATAC-seq libraries were prepared with the SureCell ATAC-seq Library Preparation kit (BioRad) and a SureCell ddSEQ Index Kit (Bio-Rad). Alignment was done with ATAC-Seq Analysis Toolkit (Bio-Rad).

For this work, we will explore two breast tumor samples (TNBC and Luminal-HER2), specifically T cells. To do so, we went to retrieve the file fragment of a sample from the [GEO \(Gene Expression Omnibus\) website](#) using the identifier provided by the author: GSE198639.

This course is done using ArchR. For more details on ArchR see [here](#).

ArchR provides a comprehensive suite for scATAC-seq analysis tools from pre-processing data to results, offering several levels of information. Moreover, ArchR is fast and ask a reasonable ressource usage.

For these analyses, you need (if you will do in your computer):

1. Install python3.6 or more:

<https://www.python.org/downloads/>

2. Install conda (miniconda or anaconda, it's a package manager for python, it allows to create python environment):

<https://conda.io/projects/conda/en/latest/user-guide/install/index.html>

3. Install macs2 package (via the terminal):

```
conda create -y -n MACS2 python=3.6
```

```
conda activate MACS2
```

```
conda install macs2 or conda install -c bioconda macs2
```

- Install of R4.1.3 or more:

<https://cran.r-project.org/>

- install these R packages (via R environment):

```
install.packages(c("devtools","BiocManager","reticulate","clustree","Seurat"))
```

```
devtools::install_github("GreenleafLab/ArchR", ref="master", repos =  
BiocManager::repositories())
```

```
ArchR::installExtraPackages()
```

`BiocManager::install("BSgenome.Hsapiens.UCSC.hg19")` (or other genome if you have data from another organism or another genome reference)

```
devtools::install_github("GreenleafLab/chromVARmotifs")
```

```
install.packages("hexbin")
```

Download preinstalled libraries and datasets

```
In [1]: # Download the installation script from GitHub  
download.file("https://github.com/eddelbuettel/r2u/raw/master/inst/scripts/add_cranapt_jammy.sh",  
              "add_cranapt_jammy.sh")  
  
# Change the script's permissions to make it executable  
Sys.chmod("add_cranapt_jammy.sh", "0755")  
  
# Execute the script in the terminal  
system("./add_cranapt_jammy.sh")
```

```
In [2]: system("apt install libfreetype6-dev libpng-dev libtiff5-dev libjpeg-dev libbz2-dev  
system("apt install libfontconfig1-dev libharfbuzz-dev libfribidi-dev libcairo2-dev  
system("apt update")  
system("apt install libmagick++-dev -y")
```

```
In [ ]: # Define a helper function to run shell commands and print their output  
shell_call <- function(command, ...) {  
  result <- system(command, intern = TRUE, ...) # Execute the command and capture output  
  cat(paste0(result, collapse = "\n")) # Print output to console  
}  
  
# Download the MACS2 package (version 2.2.9.1) using wget  
shell_call("wget https://github.com/macs3-project/MACS/archive/refs/tags/v2.2.9.1.tar.gz")  
  
# Extract the downloaded tar.gz file  
system("tar -xvf MACS.tar.gz")
```

```
# Install MACS2 in editable mode using pip
shell_call("pip install -e MACS-2.2.9.1/")
```

```
In [ ]: # Set a long timeout limit to avoid download failures
options(timeout = 1000)

# Install BiocManager if not already installed
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager", quiet = TRUE)

# Install ArchR from GitHub using devtools
devtools::install_github("GreenleafLab/ArchR", ref = "master", repos = BiocManag

# Install additional dependencies for ArchR
ArchR::installExtraPackages()

# Install other required R packages
install.packages("clustree", quiet = TRUE)
install.packages("hexbin")

# Install a specific version of the Matrix package from CRAN archives
install.packages("https://cran.r-project.org/src/contrib/Archive/Matrix/Matrix_1
```

```
In [1]: # Function to run shell commands and display the output
shell_call <- function(command, ...) {
  result <- system(command, intern = TRUE, ...) # Run command and store output
  cat(paste0(result, collapse = "\n")) # Print the output
}
```

```
In [ ]: # Set a timeout limit for downloads
options(timeout = 300)

# Download the scATAC-seq workshop dataset as a ZIP file
download.file('https://iauchile-my.sharepoint.com/:u:/g/personal/adolfo_rh_postq
'scATACseqWorkshop.zip')

# List the downloaded files with detailed information
shell_call("ls -lh")

# Unzip the downloaded file
system("unzip scATACseqWorkshop.zip")
```

NOTE: If you have errors you can do this to reanalyze the data

```
In [ ]: # Define the working directory
work_dir2 <- "/content/"
setwd(work_dir2)

# Remove any existing directory with the same name
shell_call("rm -rf scATACseqWorkshop/")

# Unzip the dataset again
shell_call("unzip scATACseqWorkshop.zip")
```

1. Define libraries, parameters and directories

Firstly, we define python library location and load R libraries. After, we define some parameters such as: 1) the number of threads we will use, 2) the working directory and 3) the location of fragment files.

In fact, ArchR can utilize multiple input formats of scATAC-seq data (the fragment files and the BAM files are the more common scATAC-seq data).

```
In [ ]: # Suppress package startup messages for cleaner output
# Load Libraries
suppressPackageStartupMessages({
  library(ArchR)
  library(reticulate)
  library(clustree)
  library(Seurat)
  library(hexbin)
})
```

```
In [ ]: # Set the Python environment path for Reticulate
Sys.setenv(RETICULATE_PYTHON = "/usr/local/bin/python")

# Check the Python configuration
py_config()

# Test if MACS2 is installed and accessible
findMac2()
```

```
In [ ]: # Set a random seed for reproducibility
set.seed(1)

# Define the number of threads to use
nb.threads = 2
addArchRThreads(threads = nb.threads)

# Set the working directory
work_dir <- "/content/scATACseqWorkshop"
setwd(work_dir)

# List and name the input fragment files
inputFiles <- list.files(file.path(work_dir, "fragments_data"), full.names = TRUE)
names(inputFiles) <- gsub("^./", "", gsub("GSM[0-9]+_", "", gsub(".fragments.ts", "")))

# Specify the reference genome for ArchR
addArchRGenome("hg19")
```

2. Create Arrow file

We create an HDF5-format Arrow file that stores all of the data associated with a sample (now and during all the analysis process). It will be updated with the additional layers of information.

If we analyse some samples, an Arrow file will be generated for each sample.

It's not a R language object and because of this, we will generate an ArchRProject object to associate the Arrow file(s) into a single analytical framework that will be rapidly accessible in R.

During this step, ArchR computes a TileMatrix containing insertion counts across genome-wide 500-bp bins (default value) and a GeneScoreMatrix that stores predicted gene expression based on weighting insertion counts in tiles nearby a gene promoter.

```
In [ ]: inputFiles

In [ ]: reformatFragmentFiles(inputFiles)

In [ ]: inputFiles <- gsub("fragments.tsv.gz", "fragments-Reformat.tsv.gz", inputFiles)

In [ ]: ArrowFiles <- createArrowFiles(
  inputFiles = inputFiles, # Input files containing scATAC-seq data
  sampleNames = names(inputFiles), # Assign sample names based on input file na
  minTSS = 0.1, # Minimum TSS enrichment score to filter low-quality cells
  minFrag = 1, # Minimum number of unique fragments per cell
  addTileMat = TRUE, # Compute and store the tile matrix for accessibility anal
  addGeneScoreMat = TRUE # Compute and store the gene score matrix for gene act
)

# Create an ArchR project using the Arrow files
project <- ArchRProject(
  ArrowFiles = ArrowFiles, # Use the generated Arrow files
  outputDirectory = "Analysis_scATACseq_noFilter", # Define output directory fo
  copyArrows = TRUE # Recommended to maintain an unaltered copy of Arrow files
)
```

If for any reason the creation of Arrow files fails (or takes too long), you can download a pre-generated version from GitHub below.

```
In [ ]: #download.file('https://github.com/integrativebioinformatics/scNotebooks/blob/ma
#download.file('https://github.com/integrativebioinformatics/scNotebooks/blob/ma
#ArrowFiles <- c('P41.arrow', 'P93.arrow')
```

Strict quality control (QC) of scATAC-seq data is essential to remove the contribution of low-quality cells.

ArchR considers three characteristics of data:

1. The fragment size (DNA fragments cut by Tn5 transposases) distribution. Due to nucleosomal periodicity, we expect to see depletion of fragments that are the length of DNA wrapped around a nucleosome (approximately 147 bp).
2. The Transcription Start Site (TSS) enrichment (signal-to-background ratio). Low signal-to-background ratio is often attributed to dead or dying cells which have de-chromatinized DNA which allows for random transposition genome-wide.
3. The number of unique nuclear fragments (i.e. not mapping to mitochondrial DNA).

We can appreciate the QC and the main metrics of samples using some plots: Plot QC metrics:

```
In [ ]: # Extract TSS enrichment and fragment count data from the ArchR project
df <- getCellColData(project, select = c("log10(nFragments)", "TSSEnrichment"))

# Create a scatter plot of TSS Enrichment vs. Log10(Unique Fragments)
plot.tss.frag <- ggPoint(
  x = df[,1], y = df[,2], # Set x-axis as log10(Unique Fragments) and y-axis as
  colorDensity = TRUE, # Color points based on density
  continuousSet = "sambaNight", # Define color theme
  xlabel = "Log10 Unique Fragments", ylabel = "TSS Enrichment", # Label axes
  xlim = c(0, quantile(df[,1], probs = 1) + 0.1), # Set x-axis limits
  ylim = c(0, quantile(df[,2], probs = 1) + 0.1) # Set y-axis limits
)

# Save the plot as a PDF in the project's "Plots" directory
plotPDF(plot.tss.frag, name = "TSS-vs-Frag.pdf", ArchRProj = project, addDOC =

# Display the plot
plot.tss.frag
```

Plot TSS metrics:

```
In [ ]: # Create a ridge plot showing TSS enrichment distribution across samples
plot.tss.v1 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample", # Group by sample
  colorBy = "cellColData", # Color based on cell metadata
  name = "TSSEnrichment", # Use TSS Enrichment as the feature to plot
  plotAs = "ridges" # Plot as ridge plot
)

# Create a violin plot with an overlaid box plot
plot.tss.v2 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample",
  colorBy = "cellColData",
  name = "TSSEnrichment",
  plotAs = "violin", # Plot as a violin plot
  alpha = 0.4, # Set transparency level
  addBoxPlot = TRUE # Overlay a box plot on top of the violin plot
)

# Display both plots side by side
plot.tss.v1 | plot.tss.v2
```

Plot fragment metrics:

```
In [ ]: # Create a ridge plot showing log10(Unique Fragments) distribution across sample
plot.frag.v1 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample",
  colorBy = "cellColData",
  name = "log10(nFragments)",
  plotAs = "ridges"
)
```

```
# Create a violin plot with an overlaid box plot
plot.frag.v2 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample",
  colorBy = "cellColData",
  name = "log10(nFrag)",
  plotAs = "violin",
  alpha = 0.4,
  addBoxPlot = TRUE
)

# Display both plots side by side
plot.frag.v1 | plot.frag.v2
```

```
In [ ]: # Create Arrow Files with Quality Filters
ArrowFiles <- createArrowFiles(
  inputFiles = inputFiles, # List of fragment files for each sample
  sampleNames = names(inputFiles), # Assign sample names based on file names
  minTSS = 4, # Minimum Transcription Start Site (TSS) enrichment score to retain
  minFrag = 1000, # Minimum number of unique fragments per cell
  addTileMat = TRUE, # Create a tile matrix for peak calling and other analyses
  addGeneScoreMat = TRUE # Compute gene activity scores
)
```

3. Doublet detection

A source of trouble in single-cell data is the contribution of "doublets" to the analysis (a doublet refers to a single droplet that received more than one nucleus). To predict which "cells" are actually doublets, ArchR synthesizes in silico doublets from the data by mixing the reads from thousands of combinations of individual cells. It projects these synthetic doublets into the UMAP embedding and identify their nearest neighbor. By iterating this procedure thousands of times, it can identify "cells" in the data whose signal looks very similar to synthetic doublets. Here, we identify the doublets.

```
In [ ]: # Compute Doublet Scores
doubletScores <- addDoubletScores(
  input = ArrowFiles, # Use Arrow files created in the previous step
  k = 10, # Number of nearest neighbors to consider for doublet detection
  knnMethod = "UMAP", # Use UMAP embedding for nearest neighbor search
  LSIMethod = 1 # Use Latent Semantic Indexing (LSI) method 1 for doublet estimation
)
```

4. Creation of ArchR project

As explain before, we generate a ArchR project to easily manipulate the scATAC-seq generated by ArchR.

```
In [ ]: # Create an ArchR Project
project <- ArchRProject(
  ArrowFiles = ArrowFiles, # Use the Arrow files created earlier
  outputDirectory = "Analysis_scATACseq", # Define output directory for the project
  copyArrows = TRUE # Keep a copy of the Arrow files for future reference
)
```

We can easily list the matrix items present in the project

```
In [ ]: # List Available Matrices in the Project
getAvailableMatrices(project) # Check what matrices (e.g., Gene Score Matrix) a
```

Strict quality control (QC) of scATAC-seq data is essential to remove the contribution of low-quality cells.

ArchR consider three characteristics of data:

1. The fragment size distribution. Due to nucleosomal periodicity, we expect to see depletion of fragments that are the length of DNA wrapped around a nucleosome (approximately 147 bp).
 2. The TSS enrichment (signal-to-background ratio). Low signal-to-background ratio is often attributed to dead or dying cells which have de-chromatized DNA which allows for random transposition genome-wide.
 3. The number of unique nuclear fragments (i.e. not mapping to mitochondrial DNA).
- We can appreciate the QC and the main metrics of samples using some plots:

Plot QC metrics:

```
In [ ]: # Plot TSS Enrichment vs. Fragment Counts
df <- getCellColData(project, select = c("log10(nFragments)", "TSSEnrichment")) # E

plot.tss.fragments <- ggPoint(
  x = df[,1], # Log10 number of unique fragments
  y = df[,2], # TSS Enrichment score
  colorDensity = TRUE, # Color by density
  continuousSet = "sambaNight", # Use the "sambaNight" color scheme
  xlabel = "Log10 Unique Fragments", # Label for X-axis
  ylabel = "TSS Enrichment", # Label for Y-axis
  xlim = c(log10(450), quantile(df[,1], probs = 1) + 0.1), # Set X-axis limits
  ylim = c(0, quantile(df[,2], probs = 1) + 0.1) # Set Y-axis limits
) +
  geom_hline(yintercept = 4, lty = "dashed", col = "black") + # Add horizontal
  geom_vline(xintercept = log10(1000), lty = "dashed", col = "black") # Add ver

# Save the plot as a PDF inside the project directory
plotPDF(plot.tss.fragments, name = "TSS-vs-Fragments.pdf", ArchRProj = project, addDOC =

# Display the plot
plot.tss.fragments
```

Plot TSS metrics:

```
In [ ]: # Plot TSS Enrichment Distributions

# Ridge plot of TSS enrichment per sample
plot.tss.v1 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample", # Group by sample
  colorBy = "cellColData", # Use cell metadata for color
  name = "TSSEnrichment", # Plot TSS enrichment scores
```



```

    plotAs = "ridges" # Use a ridge plot
  )

# Violin plot of TSS enrichment per sample with an overlaid boxplot
plot.tss.v2 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample", # Group by sample
  colorBy = "cellColData", # Use cell metadata for color
  name = "TSSEnrichment", # Plot TSS enrichment scores
  plotAs = "violin", # Use a violin plot
  alpha = 0.4, # Set transparency
  addBoxPlot = TRUE # Add a boxplot overlay
)

# Display the ridge and violin plots side by side
plot.tss.v1 | plot.tss.v2

```

Plot fragment metrics:

```

In [ ]: # Plot Fragment Count Distributions

# Ridge plot of Log10 fragment counts per sample
plot.frag.v1 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample", # Group by sample
  colorBy = "cellColData", # Use cell metadata for color
  name = "log10(nFrag)", # Plot log10 fragment counts
  plotAs = "ridges" # Use a ridge plot
)

# Violin plot of Log10 fragment counts per sample with an overlaid boxplot
plot.frag.v2 <- plotGroups(
  ArchRProj = project,
  groupBy = "Sample", # Group by sample
  colorBy = "cellColData", # Use cell metadata for color
  name = "log10(nFrag)", # Plot log10 fragment counts
  plotAs = "violin", # Use a violin plot
  alpha = 0.4, # Set transparency
  addBoxPlot = TRUE # Add a boxplot overlay
)

# Display the ridge and violin plots side by side
plot.frag.v1 | plot.frag.v2

```

Filter the doublets

```

In [ ]: # Filter Out Doublets from the Dataset
project <- filterDoublets(ArchRProj = project) # Remove detected doublets to ke

```

Sample Fragment Size Distribution and TSS Enrichment Profiles

```

In [ ]: # Plot the Transcription Start Site (TSS) enrichment profile
plot.tss.v3 <- plotTSSEnrichment(ArchRProj = project)

# Plot the fragment size distribution
plot.frag.v3 <- plotFragmentSizes(ArchRProj = project)

```

```
# Display both plots side by side  
plot.frag.v3 | plot.tss.v3
```

5. Normalization, dimensional reduction, batch effect correction, clustering and others step

5.1. Normalization and dimensional reduction

scATAC-seq generates a sparse insertion counts matrix (500-bp tiles; binary data of ~6 million of features) making it impossible to identify variable peaks for standard dimensionality reduction. To get around this issue, ArchR use LSI (Latent Semantic Indexing), a layered dimensionality reduction approach for sparse and noisy data.

Rather than identifying the most variable peaks, ArchR tries using the most accessible features as input to LSI.

However, when running multiple samples the results could shown high degrees of noise and low reproducibility.

To remedy this, ArchR introduced the "iterative LSI" approach (Satpathy, Granja et al., 2019), which computes an initial LSI transformation on the most accessible tiles and identifies lower resolution clusters that are not batch confounded.

1 - This approach computes an initial LSI transformation on the most accessible tiles and identifies lower resolution clusters that are not batch confounded.

2 - ArchR computes the average accessibility for each of these clusters across all features. ArchR then identifies the most variable peaks across these clusters and uses these features for LSI again.

3 - In this second iteration, the most variable peaks are more similar to the variable genes used in scRNA-seq LSI implementations.

This approach minimizes observed batch effects and allow dimensionality reduction operations on a more reasonably sized feature matrix.

```
In [ ]: project_Normalized <- addIterativeLSI(ArchRProj = project, iterations = 2,  
      # Number of iterations for LSI; more itera  
      #sampleCellsPre = 50000, # Optional: Numbe  
      #clusterParams = list(resolution = 0.1, sa  
      # Cluster parameters can be adjusted to op  
      useMatrix = "TileMatrix", # Use TileMatrix  
      name = "IterativeLSI", # Name of the reduc  
      varFeatures = 25000) # Number of variable
```

5.2. Batch effect correction

Sometimes the iterative LSI approach isn't enough to correct strong batch effect differences. For this reason, ArchR implements a commonly used batch effect correction tool called Harmony (Korsunsky et al., 2019) which was originally designed for scRNA-seq.

[illegible]

5.3. UMAP

Run UMAP in ArchR

```
In [ ]: # Compute UMAP embedding based on Iterative LSI dimensions
project_Normalized <- addUMAP(ArchRProj = project_Normalized, reducedDims = "Iterative LSI")

# Plot UMAP colored by sample identity
plotEmbedding(ArchRProj = project_Normalized, colorBy = "cellColData", name = "Sample")

# Compute UMAP embedding based on Harmony dimensions (after batch correction)
project_Normalized <- addUMAP(ArchRProj = project_Normalized, reducedDims = "Harmony")

# Plot UMAP again to visualize batch-corrected embedding
plotEmbedding(ArchRProj = project_Normalized, colorBy = "cellColData", name = "Sample")
```

5.4. Clustering

To identify clusters, ArchR allows to use same method as Seurat or Scrn. We have selected the same method describe the first day and used in Seurat package.

```
In [ ]: # Iterate over different clustering resolutions from 0.0 to 0.9
for(i in seq(0,0.9,0.1)){
  project_Normalized <- addClusters(input = project_Normalized, reducedDims = "H
    method = "Seurat", # Clustering method (Seurat)
    name = paste("Clusters.res",i,sep=""), # Name of the clusters
    resolution = i, # Set clustering resolution
    verbose = FALSE) # Suppress verbose output
}
```

5.5. Save and load a project

Save

```
In [ ]: # Save the current state of the ArchR project to disk
saveArchRProject(ArchRProj = project_Normalized,
                  outputDirectory = file.path(getwd(), "Analysis scATACseq"))
```

Load

[illegible]

6. Exploration of data using Gene estimation

6.1. Visualization of clustering using Clustree

Clustree is an useful tool to explore the links between the clusters from different resolutions.

```
In [ ]: # Extract clustering information from ArchR project
tmp.clustree.datatable <- as.data.frame(project_Normalized@cellColData)

# Plot a clustering tree to visualize clustering relationships across resolution
clustree(tmp.clustree.datatable, prefix="Clusters.res")
```

6.2. Visualization of clustering on UMAP

```
In [ ]: # Iterate over different clustering resolutions and visualize UMAP embeddings
for(i in seq(0,0.9,0.1)){
  # Plot UMAP with cluster labels
  plot.umap.resi <- plotEmbedding(ArchRProj = project_Normalized,
                                colorBy = "cellColData",
                                name = paste("Clusters.res",i,sep=""),
                                embedding = "UMAP", size=0.1)

  # Plot UMAP without cluster labels
  plot.umap.wolabel.resi <- plotEmbedding(ArchRProj = project_Normalized,
                                          colorBy = "cellColData",
                                          name = paste("Clusters.res",i,sep=""),
                                          embedding = "UMAP", size=0.1,
                                          labelMeans=FALSE)

  # Display both plots side by side
  print(plot.umap.resi | plot.umap.wolabel.resi)
}
```

6.3. Characterization of clusters

At this step, we select a specific resolution to explore in details the GeneScore to characterize the different clusters. For that, we will identify marker genes (based on gene scores, or estimation of gene expression) of clusters. In short, ArchR estimates gene scores using the local accessibility of the gene region that includes the promoter and gene body, but imposes an exponential weight that accounts for the activity of putative distal regulatory elements as a function of distance.

 from ArchR manual{width=70% height=50%}

Remarks: ArchR can used gene, peak or transcription factor motif features. Here, ArchR identify the genes that appear to be uniquely active in each cluster at the resolution 0.4.

```
In [ ]: slct.res = "res0.7" # Select resolution for analysis

# Identify marker genes using Gene Score Matrix
```



```
# Display selected genes
plot.GS.woMAGIC$VIM | plot.GS.woMAGIC$EPCAM
```

However, scATAC-seq data is really sparse. Due to that, it is highly suggest to use MAGIC (van Dijk, et al., 2018), which add an imputation weight to the gene scores by smoothing signal across nearby cells.

```
In [ ]: # Apply MAGIC for gene imputation
project_Normalized <- addImputeWeights(project_Normalized)

# Plot Gene Score UMAP with imputation
plot.GS <- plotEmbedding(ArchRProj = project_Normalized, colorBy = "GeneScoreMat
                        name = markerGenes,
                        embedding = "UMAP",
                        imputeWeights = getImputeWeights(project_Normalized))

plot.GS$VIM | plot.GS$EPCAM
plot.GS$FLT4 | plot.GS$THY1
plot.GS$ITGAX | plot.GS$CD14
plot.GS$MS4A1 | plot.GS$CD38
plot.GS$CD3D | plot.GS$CD8A
plot.GS$CD4 | plot.GS$GZMA
```

7. scATAC-scRNAseq integration

ArchR enables integration with scRNA-seq, offers the possibility to use clusters called in scRNA-seq space or use the gene expression measurements after integration.

The way this integration works is by directly aligning cells from scATAC-seq with cells from scRNA-seq by comparing the scATAC-seq gene score matrix with the scRNA-seq gene expression matrix. This alignment is performed using the FindTransferAnchors() function from the Seurat package which allows you to align data across two datasets.

However, to appropriately scale this procedure for hundreds of thousands of cells ArchR provides, a parallelization of this procedure by dividing the total cells into smaller groups of cells and performing separate alignments.

```
In [31]: # Import scRNAseq data
scRNA<-readRDS(file.path(work_dir,"scRNAseq.data.rds"))
DefaultAssay(object = scRNA) <- "RNA"

# Integrate scRNA-seq and scATAC-seq data
project_Normalized <- addGeneIntegrationMatrix(ArchRProj = project_Normalized,
  useMatrix = "GeneScoreMatrix", matrixName = "GeneIntegrationMatrix",
  reducedDims = "Harmony", #Harmony , IterativeLSI
  seRNA = scRNA, addToArrow = TRUE, force= TRUE,
  groupRNA = "integrated_snn_res.0.5",
  nameCell = "predictedCell", nameGroup = "predictedGroup", nameScore = "predi
  sampleCellsATAC = 10000, sampleCellsRNA = 10000, scaleTo = 10000)
project_Normalized <- addImputeWeights(project_Normalized)

saveArchRProject(ArchRProj = project_Normalized, load = FALSE)

plot_rna.woLabel <- plotEmbedding(project_Normalized,colorBy = "cellColData",nam
```

```
plot_rna <- plotEmbedding(project_Normalized,colorBy = "cellColData",name = "pre

# Cross table between scRNA-seq and scATAC-seq data
cM <- as.matrix(confusionMatrix(project$Clusters.res0.7,           # Warnin
                               project$predictedGroup))           # Warnin
```

8. Peak calling

Calling peaks is one of the most fundamental processes in ATAC-seq data analysis. Because per-cell scATAC-seq data is essentially binary (accessible or not accessible), we perform calling peaks on groups of similar cells (or clusters) define previously.

ArchR applies a Iterative Overlap Peak Merging Procedure with the recommended MACS2 peak caller (Zhang et al., 2008).

It uses a function to perform this iterative overlap peak merging procedure:

1. ArchR would call peaks for each pseudo-bulk replicate individually.
2. ArchR would analyze all of the pseudo-bulk replicates from a single cell type together, performing the first iteration of iterative overlap removal.
3. After the first iteration of iterative overlap removal, ArchR checks to see the reproducibility of each peak across pseudo-bulk replicates and only keeps peaks that pass a threshold indicated by the reproducibility parameter.
4. At the end of this process, we would have a single merged peak set for each cell types.

8.1. Creation of the pseudo-bulk replicates

```
In [ ]: # Determine the number of replicates to use for coverage calculation
nbReplicates = ifelse(length(names(table(project_Normalized$Sample))) > 5,
                      length(names(table(project_Normalized$Sample))), 5)

# Add group coverage information to the ArchR project
project_Peaks <- addGroupCoverages(
  ArchRProj = project_Normalized,
  maxReplicates = nbReplicates,
  groupBy = paste("Clusters", slct.res, sep = ".")
)
```

8.2. Perform peak calling

```
In [ ]: # Find the path to MACS2
pathToMacs2 <- findMacs2()

# Perform peak calling using MACS2
project_Peaks <- addReproduciblePeakSet(
  ArchRProj = project_Peaks,
  groupBy = paste("Clusters", slct.res, sep = "."),
  pathToMacs2 = pathToMacs2
```

```
)

# Alternative peak calling method (use if MACS2 is unavailable)
# project_Peaks <- addReproduciblePeakSet(
#   ArchRProj = project_Peaks,
#   groupBy = paste("Clusters", slct.res, sep = "."),
#   peakMethod = "Tiles",
#   method = "p"
# )

# Retrieve the peak set after calling peaks
getPeakSet(project_Peaks)

# Add imputation weights to improve downstream analyses
project_Peaks <- addImputeWeights(project_Peaks)

# Save the ArchR project with peak calling results
saveArchRProject(
  ArchRProj = project_Peaks,
  outputDirectory = file.path(getwd(), "Analysis_scATACseq"),
  load = TRUE
)
```

8.3. Identification of marker peaks

As explained before for the marker genes, ArchR can use gene, peak or transcription factor motif features. Here, ArchR identifies the peaks that appear to be uniquely active in each cluster at selected resolution.

```
In [ ]: # Generate a Peak Matrix for accessibility quantification
project_Peaks <- addPeakMatrix(project_Peaks)

# Identify marker peaks for each cluster using Wilcoxon test
markersPeaks <- getMarkerFeatures(
  ArchRProj = project_Peaks,
  useMatrix = "PeakMatrix",
  groupBy = paste("Clusters", slct.res, sep = "."),
  bias = c("TSSEnrichment", "log10(nFragments)"),
  testMethod = "wilcoxon"
)

# Extract significantly different peaks (FDR <= 0.01, Log2FC >= 1)
markerList <- getMarkers(markersPeaks, cutOff = "FDR <= 0.01 & Log2FC >= 1")

# View marker peaks for cluster C9
markerList[["C9"]]
```

To visualize the marker genes, we can produce a heatmap:

```
In [ ]: # Generate a heatmap of marker peaks (less stringent threshold)
heatmapPeaks <- plotMarkerHeatmap(
  seMarker = markersPeaks,
  cutOff = "FDR <= 0.1 & Log2FC >= 0.5",
  transpose = FALSE
)
```



```
# Display the heatmap
heatmapPeaks
```

Or MA and volcano plots of marker peaks by cluster:

```
In [ ]: # Generate MA and Volcano plots for cluster C9
map <- plotMarkers(
  seMarker = markersPeaks,
  name = "C9",
  cutOff = "FDR <= 0.1 & Log2FC >= 1", # Default
  plotAs = "MA"
)

vp <- plotMarkers(
  seMarker = markersPeaks,
  name = "C9",
  cutOff = "FDR <= 0.1 & Log2FC >= 1", # Default
  plotAs = "Volcano"
)

# Combine both plots
map | vp
```

Or visualize the marker peaks on a browser track:

```
In [ ]: # Generate a browser track visualization for CD4
plot.track1 <- plotBrowserTrack(
  ArchRProj = project_Peaks,
  groupBy = paste("Clusters", slct.res, sep = "."),
  geneSymbol = c("CD4"),
  features = getMarkers(markersPeaks, cutOff = "FDR <= 0.1 & Log2FC >= 1", ret
    upstream = 50000, downstream = 50000
  )

# Display the track
grid::grid.newpage()
grid::grid.draw(plot.track1$CD8A)

In [38]: # Save the object and download it!!
saveRDS(project_Peaks, "project_Peaks.rds")
saveRDS(markersPeaks, "markersPeaks.rds")
```

9. Motif Enrichment

After identified peak sets, the next step is to predict what transcription factors (TFs) may be mediating the binding events that create those accessible chromatin sites.

ArchR allow to annotate the TF motifs that are enriched in peaks that are up or down in the different cell types.

Firstly, we add motif annotations to our ArchRProject based on a reference database (for example: CIS-BP, JASPAR, Encode or Homer).

Here, we have selected CIS-BP which contains > 300 TF families from > 700 species collecting from > 70 sources , including other databases: Transfac, JASPAR, Hocomoco,

FactorBook, UniProbe, among others.

Next, we test the set of significantly differential peaks for motif enrichment.

```
In [ ]: # Download and reload data if needed
shell_call("gdown 1SdSmF9R3yHNWacmFf22RxprrKmUHM_b")
markersPeaks = readRDS("markersPeaks.rds")

shell_call("gdown 1S6fRM7_KX4kd9ankvzA5HloJJSIM4bn")
project_Peaks = readRDS("project_Peaks.rds")

In [ ]: ## Motif Enrichment
project_Peaks <- addMotifAnnotations(ArchRProj = project_Peaks, motifSet = "cisb")

# Motif enrichment in marker peaks
enrichMotifs <- peakAnnoEnrichment(
  seMarker = markersPeaks, ArchRProj = project_Peaks,
  peakAnnotation = "Motif", cutOff = "FDR <= 0.1 & Log2FC >= 0.5") # Def

# You have two output, the Enrichment matrix and the pvalue matrix:
head(enrichMotifs@assays@data$Enrichment, 10)
```

We can display a heatmap to visualize the main motifs of each clusters.

```
In [ ]: # Plot a heatmap of enriched motifs
heatmapEM <- plotEnrichHeatmap(enrichMotifs, n = 7, transpose = TRUE)
ComplexHeatmap::draw(heatmapEM, heatmap_legend_side = "bot", annotation_legend_s
```

9.1. ChromVAR and visualization of motif deviation

ChromVAR, developed by Greenleaf lab, is designed for predicting enrichment of TF activity on a per-cell basis from sparse chromatin accessibility data. ChromVAR computes:

1. A “deviation”, which is a bias-corrected measurement of how far the per-cell accessibility of a given feature (i.e motif) deviates from the expected accessibility based on the average of all cells or samples.
2. A “z-score” / a “deviation score”, which is the z-score for each bias-corrected deviation across all cells.

```
In [ ]: # Add background peaks
project_Peaks <- addBgdPeaks(project_Peaks)

# Compute deviations matrix
project_Peaks <- addDeviationsMatrix(ArchRProj = project_Peaks, peakAnnotation =

# Plot variability in motif accessibility
getVarDeviations(project_Peaks, name = "MotifMatrix", plot = TRUE)

# Save project
saveArchRProject(ArchRProj = project_Normalized,
  outputDirectory = file.path(getwd(), "Analysis_scATACseq"))
```

We can display a distribution of markers

```
In [ ]: # Define a list of motifs to analyze
motifs <- c("FOS", "JUNB")

# Retrieve motif features from the MotifMatrix that match the selected motifs
markerMotifs <- getFeatures(
  project_Peaks,
  select = paste(motifs, "_", collapse = "|", sep = ""),
  useMatrix = "MotifMatrix"
)

# Filter motif features to include only those with the "z:" prefix
markerMotifs <- grep("z:", markerMotifs, value = TRUE)

# Add imputation weights to the ArchR project for smoothing data visualization
project_Peaks <- addImputeWeights(project_Peaks)

# Generate grouped motif enrichment plots
cowp <- plotGroups(
  ArchRProj = project_Peaks,
  groupBy = paste("Clusters", slct.res, sep = "."),
  colorBy = "MotifMatrix",
  name = markerMotifs,
  imputeWeights = getImputeWeights(project_Peaks)
)

# Arrange plots in a grid layout with two columns
do.call(cowplot::plot_grid, c(list(ncol = 2), cowp))
```

Or visualize the motif deviation in UMAP (and see if motif deviation correlates with TF gene score)

```
In [ ]: # Plot motif enrichment on UMAP embedding
motif.umap <- plotEmbedding(
  ArchRProj = project_Peaks,
  colorBy = "MotifMatrix",
  name = sort(markerMotifs),
  embedding = "UMAP",
  imputeWeights = getImputeWeights(project_Peaks)
)

# Display motif UMAP plots in a grid layout
do.call(cowplot::plot_grid, c(list(ncol = 2), motif.umap))

# Retrieve gene activity features related to the selected motifs
markerRNA <- getFeatures(
  project_Peaks,
  select = paste(motifs, "$", collapse = "|", sep = ""),
  useMatrix = "GeneScoreMatrix"
)

# Plot gene score matrix enrichment on UMAP embedding
gene.umap <- plotEmbedding(
  ArchRProj = project_Peaks,
  colorBy = "GeneScoreMatrix",
  name = sort(markerRNA),
  embedding = "UMAP",
  imputeWeights = getImputeWeights(project_Peaks)
)
```

```
# Display gene score UMAP plots in a grid layout
do.call(cowplot::plot_grid, c(list(ncol = 2), gene.umap))
```

9.2. Pairwise test between clusters

We can identify the motif enrichment between two clusters (based on differential accessibility of peaks between these two clusters).

```
In [ ]: slct.Cl1="C9"
slct.Cl2="C11"

# Perform differential analysis between C9 and C11
markerTest <- getMarkerFeatures(ArchRProj = project_Peaks,
                                useMatrix = "PeakMatrix",
                                groupBy = paste("Clusters",slct.res,sep="."),
                                testMethod = "wilcoxon",
                                bias = c("TSSEnrichment", "log10(nFragments)"),
                                useGroups = slct.Cl1, bgdGroups = slct.Cl2)

# Generate MA and Volcano plots
map.Cl1vCl2 <- markerPlot(seMarker = markerTest, name = slct.Cl1,
                          cutOff = "FDR <= 0.1 & abs(Log2FC) >= 1",
                          plotAs = "MA")
vp.Cl1vCl2 <- markerPlot(seMarker = markerTest, name = slct.Cl1,
                          cutOff = "FDR <= 0.1 & abs(Log2FC) >= 1",
                          plotAs = "Volcano")

map.Cl1vCl2 | vp.Cl1vCl2
```

Motif Up-Enrich and motif Down-enrich (based on the pairwise test between clusters)

```
In [ ]: # Identify significantly enriched motifs in peaks with increased accessibility
motifsUp <- peakAnnoEnrichment(ArchRProj = project_Peaks,
                               seMarker = markerTest,
                               peakAnnotation = "Motif",
                               cutOff = "FDR <= 0.1 & Log2FC >= 0.5") # Select m

# Create a data frame with motif names and -log10 adjusted p-values
df <- data.frame(TF = rownames(motifsUp), mlog10Padj = assay(motifsUp)[,1])
df <- df[order(df$mlog10Padj, decreasing = TRUE),] # Sort by significance
df$rank <- seq_len(nrow(df)) # Assign rank based on significance

# Plot enriched TF motifs with Labels
ggUp <- ggplot(df, aes(rank, mlog10Padj, color = mlog10Padj)) +
  geom_point(size = 1) + ggrepel::geom_label_repel(
    data = df[rev(seq_len(30)), ], aes(x = rank, y = mlog10Padj, label = TF),
    size = 1.5, nudge_x = 2, color = "black") + theme_ArchR() +
  ylab("-log10(P-adj) Motif Enrichment") + xlab("Rank Sorted TFs Enriched")
  scale_color_gradientn(colors = paletteContinuous(set = "comet"))

# Identify significantly enriched motifs in peaks with decreased accessibility
motifsDo <- peakAnnoEnrichment(ArchRProj = project_Peaks,
                               seMarker = markerTest,
                               peakAnnotation = "Motif",
                               cutOff = "FDR <= 0.1 & Log2FC <= -0.5") # Select

df <- data.frame(TF = rownames(motifsDo), mlog10Padj = assay(motifsDo)[,1])
```

```
df <- df[order(df$mlog10Padj, decreasing = TRUE),] # Sort by significance
df$rank <- seq_len(nrow(df)) # Assign rank

# Plot TF motifs that are lost in accessibility
ggDo <- ggplot(df, aes(rank, mlog10Padj, color = mlog10Padj)) +
  geom_point(size = 1) + ggrepel::geom_label_repel(
    data = df[rev(seq_len(30)), ], aes(x = rank, y = mlog10Padj, label = TF),
    size = 1.5, nudge_x = 2, color = "black") + theme_ArchR() +
    ylab("-log10(FDR) Motif Enrichment") + xlab("Rank Sorted TFs Enriched")
    scale_color_gradientn(colors = paletteContinuous(set = "comet"))

# Combine both plots
ggUp | ggDo
```

10. Identification of Positive TF-Regulators

Although ATAC-seq allows unbiased identification of TFs, families of TFs share similar motifs when viewed in aggregate. This makes it difficult to identify specific TFs that may be responsible for the observed changes in chromatin accessibility to their predicted binding sites. To circumvent this problem, ArchR identifies TFs whose gene expression (Gene score) is positively correlated with changes in accessibility of their corresponding motif (Motif deviation obtained using ChromVAR).

Step 1. Identify Deviant TF Motifs

```
In [ ]: # Extract motif deviation scores grouped by clusters
seGroupMotif <- getGroupSE(ArchRProj = project_Peaks, useMatrix = "MotifMatrix",

# Extract only Z-score deviations
seZ <- seGroupMotif[rowData(seGroupMotif)$seqnames == "z",]

# Compute the maximum delta in Z-score across all clusters
rowData(seZ)$maxDelta <- lapply(seq_len(ncol(seZ)), function(x){
  rowMaxs(assay(seZ) - assay(seZ)[,x])
}) %>% Reduce("cbind", .) %>% rowMaxs
```

Step 2. Identify Correlated TF Motifs and TF Gene Score/Expression

```
In [ ]: # Compute correlations between gene scores and motif deviations
corGSM_MM <- correlateMatrices(
  ArchRProj = project_Peaks,
  useMatrix1 = "GeneScoreMatrix",
  useMatrix2 = "MotifMatrix",
  reducedDims = "Harmony" # Can also use IterativeLSI
)

# Display top correlations
head(corGSM_MM, 15)
```

Step 3. Add Maximum Delta Deviation to the Correlation Data Frame

```
In [ ]: # Annotate motifs with the max delta observed between clusters
corGSM_MM$maxDelta <- rowData(seZ)[match(corGSM_MM$MotifMatrix_name, rowData(seZ
```

Step 4. Identify Positive TF Regulators

```
In [ ]: # Sort by absolute correlation and remove duplicate motifs
corGSM_MM <- corGSM_MM[order(abs(corGSM_MM$cor), decreasing = TRUE), ]
corGSM_MM <- corGSM_MM[which(!duplicated(gsub("\\-.*", "", corGSM_MM["MotifMatr

# Classify TFs as positive (PLUS), negative (NEG), or neutral (NO)
corGSM_MM$TFRegulator <- "NO"
corGSM_MM$TFRegulator[which(corGSM_MM$cor > 0.1 & corGSM_MM$padj < 0.01 & corGSM
corGSM_MM$TFRegulator[which(corGSM_MM$cor < (-0.1) & corGSM_MM$padj < 0.01 & cor

# Scatter plot of correlation vs max delta
ggplot(data.frame(corGSM_MM), aes(cor, maxDelta, color = TFRegulator)) +
  geom_point() +
  theme_ArchR() +
  geom_vline(xintercept = 0, lty = "dashed") +
  scale_color_manual(values = c("NO"="darkgrey", "PLUS"="firebrick3", "NEG"="roy
  xlab("Correlation To Gene Score") +
  ylab("Max TF Motif Delta") +
  scale_y_continuous(
    expand = c(0,0),
    limits = c(0, max(corGSM_MM$maxDelta)*1.05)
  )

# Display top regulators
head(as.matrix(sort(corGSM_MM[corGSM_MM$TFRegulator=="PLUS", c("GeneScoreMatrix_
head(as.matrix(sort(corGSM_MM[corGSM_MM$TFRegulator=="NEG", c("GeneScoreMatrix_n
```

11. Co-accessibility

To study how the genes are regulated (promoter and enhancer links) ArchR proposes Co-accessibility analysis. Co-accessibility is a correlation in accessibility between two peaks across many single cells. Said another way, when Peak A is accessible in a single cell, Peak B is often also accessible. For example, co-accessibility allows to visualize the enhancer(s) linked to the gene promoter.

from ArchR manual{width=50% height=50%}

Remarks:Co-accessibility analysis identify cell type-specific peaks. Although these peaks are often accessible together within a single cell type (and often all not accessible in all other cell types) does not necessarily explain a regulatory relationship between these peaks.

```
In [ ]: # Add co-accessibility analysis to the ArchR project using Harmony dimensions
project_Peaks <- addCoAccessibility(ArchRProj = project_Peaks, reducedDims = "Ha

# Retrieve co-accessibility interactions with correlation cutoff and resolution
cA <- getCoAccessibility(
  ArchRProj = project_Peaks,
  corCutOff = 0.5,
  resolution = 10000,
```

```

returnLoops = TRUE
)

# Display the first 10 co-accessibility interactions
head(cA$CoAccessibility, 10)

# Generate a genome browser track visualization for selected marker genes
p <- plotBrowserTrack(
  ArchRProj = project_Peaks,
  groupBy = paste("Clusters", slct.res, sep = "."),
  geneSymbol = markerGenes,
  upstream = 50000, # Extend 50kb upstream
  downstream = 50000, # Extend 50kb downstream
  loops = getCoAccessibility(project_Peaks)
)

# Render the genome browser plot
grid::grid.newpage()
grid::grid.draw(p$CD8A)

```

12. Footprinting

Transcription factor (TF) footprinting allows for the prediction of the precise binding location of a TF at a particular locus. This is because the DNA bases that are directly bound by the TF are actually protected from transposition while the DNA bases immediately adjacent to TF binding are accessible.

```

In [ ]: # Get motif positions
motifPositions <- getPositions(project_Peaks)

# Remove 'z:' prefix from motif names
markerMotifs <- gsub("z:", "", markerMotifs)

# Compute motif footprints
seFoot <- getFootprints(
  ArchRProj = project_Peaks,
  positions = motifPositions[markerMotifs],
  groupBy = paste("Clusters", slct.res, sep=".")
)

# Plot footprints with bias correction
plotFootprints(seFoot = seFoot,
  ArchRProj = project_Peaks,
  normMethod = "Subtract", # Options: Divide, None
  plotName = paste("Footprints-Subtract-Bias", slct.res, "cisbp", s
  addDOC = FALSE,
  smoothWindow = 5)

```

13. Trajectory Analysis

ArchR proposes to create a cellular trajectory that approximates the differentiation from a cell cluster to an other one. After the definition of the trajectory backbone, which consist of an ordered vector of cell group labels, ArchR identify a pseudo-time value for each cell in the trajectory. In the results, ArchR provides UMAPs to visualize the pseudo-

temporal trajectory and heatmaps to track spike/pattern/gene signals as a function of pseudo-temporality.

13.1. Construction of trajectory

Firstly, ArchR produces a pseudo-time value for each cell in the trajectory, which can be visualize on UMAP and used to display an arrow approximating the trajectory path from the spline-fit.

```
In [ ]: # Define a trajectory (e.g., C9 to C11)
trajectory <- c("C9", "C11")
traj.name <- "TF.C9.C11"

# Add trajectory to the project
project_Peaks <- addTrajectory(
  ArchRProj = project_Peaks,
  name = traj.name,
  groupBy = paste("Clusters", slct.res, sep="."),
  trajectory = trajectory,
  embedding = "UMAP",
  force = TRUE
)

# Plot trajectory
plotTraj <- plotTrajectory(project_Peaks, trajectory = traj.name, colorBy = "cell",
plotTraj[[1]]
```

13.2. Observation of specific genes

It's possible to visualize this trajectory but color the cells by a specific gene score value.

```
In [ ]: # Plot trajectory of the gene CD4 using the GeneScoreMatrix, visualized in UMAP
p_gene <- plotTrajectory(project_Peaks, trajectory = traj.name, colorBy = "GeneScoreMatrix",
name = "CD4", continuousSet = "horizonExtra", embedding

# Display the first two trajectory plots side by side
p_gene[[1]] | p_gene[[2]]
```

13.3 Pseudo-time heatmaps

Finally, ArchR allow to perform heatmap to visualize changes in many features (peaks, gene scores or motifs) across pseudo-time.

```
In [ ]: # Obtain the gene score trajectory (normalized with Log2)
trajGSM <- getTrajectory(ArchRProj = project_Peaks, name = traj.name, useMatrix = "GeneScoreMatrix")

# Plot heatmap for gene score trajectory with "horizonExtra" color palette
p_trajGSM <- plotTrajectoryHeatmap(trajGSM, pal = paletteContinuous(set = "horizonExtra"))

# Generate heatmap matrix for gene score trajectory
p_trajGSM.matrix <- plotTrajectoryHeatmap(trajGSM, pal = paletteContinuous(set = "horizonExtra"))

# Obtain the peak accessibility trajectory (normalized with Log2)
trajPM <- getTrajectory(ArchRProj = project_Peaks, name = traj.name, useMatrix = "PeakAccessibility")
```



```

# Plot heatmap for peak accessibility trajectory with "solarExtra" color palette
p_trajPM <- plotTrajectoryHeatmap(trajPM, pal = paletteContinuous(set = "solarEx

# Generate heatmap matrix for peak accessibility trajectory
p_trajPM.matrix <- plotTrajectoryHeatmap(trajPM, pal = paletteContinuous(set = "

# Obtain the motif activity trajectory (without log2 normalization)
trajMM <- getTrajectory(ArchRProj = project_Peaks, name = traj.name, useMatrix

# Plot heatmap for motif activity trajectory with "solarExtra" color palette
p_trajMM <- plotTrajectoryHeatmap(trajMM, pal = paletteContinuous(set = "solarEx

# Generate heatmap matrix for motif activity trajectory
p_trajMM.matrix <- plotTrajectoryHeatmap(trajMM, pal = paletteContinuous(set = "

# Display the heatmaps
p_trajGSM
p_trajPM
p_trajMM

```

13.3. Integrative pseudo-time analysis

As shown before, ArchR allows also to perform integrative analysis to identify positive TF using gene scores and motif accessibility across pseudo-time, follow their variability across pseudo-time and understand their role in this trajectory. To do it, ArchR proposes to use the `correlateTrajectories()` function which takes two `SummarizedExperiment` objects obtained from the `getTrajectories()` function which we have obtained before.

- Step 1. Identify and select the motifs which gene score and TF motif accessibility is correlated:

```

In [ ]: # Compute correlation between gene score trajectory (trajGSM) and motif trajectory
# Using low stringency criteria: correlation cutoff of 0.2, variance cutoffs of
corGSM_MM <- correlateTrajectories(trajGSM, trajMM,
                                   corCutoff = 0.2, varCutoff1 = 0.5, varCutoff2

# Filter the gene score and motif trajectories based on correlation results
flt_trajGSM <- trajGSM[corGSM_MM[[1]]$name1, ]
flt_trajMM <- trajMM[corGSM_MM[[1]]$name2, ]

```

- Step 2. Create a new trajectory to visualize side by side the TF motif based on gene score and TF motif enrichment:

```

In [ ]: # Create a combined trajectory object using the filtered gene score trajectory (
combinedTraj <- flt_trajGSM

# Normalize and combine the gene score trajectory (flt_trajGSM) and motif trajectory
# - Scale each row (gene/motif) separately for both matrices
# - Transpose the result and add them together to integrate information from both
assay(combinedTraj, withDimnames=FALSE) <- t(apply(assay(flt_trajGSM), 1, scale))
                                     t(apply(assay(flt_trajMM), 1, scale))

# Generate a heatmap matrix from the combined trajectory
# - returnMat = TRUE returns the matrix instead of plotting

```

```
# - varCutOff = 0 ensures no variance-based filtering
combinedMat <- plotTrajectoryHeatmap(combinedTraj, returnMat = TRUE, varCutOff = 0)

# Determine the order of rows (genes/motifs) in the combined matrix
rowOrder <- match(rownames(combinedMat), rownames(flt_trajGSM))

# Plot heatmap for the gene score trajectory, keeping row order consistent with the combined matrix
ht_GSM <- plotTrajectoryHeatmap(flt_trajGSM, pal = paletteContinuous(set = "horizontal", varCutOff = 0, rowOrder = rowOrder))

# Plot heatmap for the motif trajectory, keeping row order consistent with the combined matrix
ht_MM <- plotTrajectoryHeatmap(flt_trajMM, pal = paletteContinuous(set = "solarEclipse", varCutOff = 0, rowOrder = rowOrder))

# Display both heatmaps side by side for comparison
ht_GSM + ht_MM
```

Session Information

```
In [ ]: # Display session info to track package versions
sessionInfo()
```